

Analyzing the Role of AI Assisted Programming in Modern Software Development

Assessment 2 – AI Assisted Programming

Module: Advanced Programming Concepts (IT3003)

Student: M.K.P. Fernando

Student ID: s17234

Registration No: 23s19990

Selected Problem: Library Management System

Table of Contents

Table of Contents.....	2
Introduction.....	3
Conceptual Understanding.....	3
What is AI-Assisted Programming?	3
Types of AI Coding Tools	4
Advantages in Software Development.....	4
Risks and Limitations	5
Practical Implementation	6
Version A - Traditional Approach (LibrarySystemA.java)	6
Version B - AI-Assisted Approach (Java package library)	13
Comparison of Version A and Version B	20
UML Diagrams	21
AI Prompts Used During Development.....	25
Critical Analysis.....	26
How AI Changed the Coding Approach	26
Did AI Improve or Reduce Understanding?	26
When AI Should NOT Be Used.....	26
Ethical Considerations in AI-Generated Code.....	27
Conclusion	28
References.....	29
Appendix: GitHub Repository Submission	30

Introduction

Artificial Intelligence is becoming a core part of modern software development, with tools like AI code generators, smart debuggers, and intelligent programming assistants reshaping the way developers build and maintain applications. These technologies are changing how software is designed, tested, and supported.

This report was prepared for Assessment 2 of the Advanced Programming Concepts module, which asks students to explore the impact of AI assisted programming in software engineering, considering both its technical benefits and ethical challenges.

To address this, a Library Management System was built twice:

- Version A – using a traditional, fully manual coding approach.
- Version B – developed with AI assisted workflows.

The two versions are compared in terms of development speed, code quality, error handling, and readability/maintainability, with supporting UML diagrams and console outputs.

The report concludes with a critical reflection on AI assisted programming and offers a final perspective on its role in both education and the wider software industry.

Conceptual Understanding

What is AI Assisted Programming?

AI assisted programming means using machine learning models most often large language models (LLMs) trained on huge collections of source code and natural language to support developers in writing, debugging, testing, and documenting software. Instead of coding every line manually, developers can work alongside AI assistants like GitHub Copilot, ChatGPT, or Claude, which integrate directly into editors, terminals, or chat interfaces. These assistants generate code suggestions, explanations, or even complete functions based on natural language prompts and the surrounding code context.

Importantly, the developer still reviews, tests, and integrates the output. That makes AI assisted programming less about full automation and more about collaboration where human judgment and machine generated suggestions combine to improve productivity and code quality.

Types of AI Coding Tools

- Code generation tools - produce new code, boilerplate, or entire functions/classes from a natural language description (e.g - GitHub Copilot, ChatGPT, Claude, Amazon Q Developer).
- Debugging assistants - analyze stack traces, error messages, or failing tests and suggest root causes and fixes
- Refactoring tools - restructure existing code for readability, performance, or design-pattern conformance without changing behaviour.
- Code review - flag code smells, security issues, or style violations and propose corrections.
- Test generation tools - generate unit tests and edge cases from existing implementation code.

Advantages in Software Development

- Faster delivery of boilerplate and repetitive code (getters/setters, data classes, CRUD scaffolding), freeing developer time for business logic and design decisions.
- Lower barrier to entry for unfamiliar APIs, libraries, or language features, since the assistant can explain and demonstrate usage inline.
- Improved consistency - AI suggestions tend to follow common conventions (naming, exception handling, javadoc), which can raise the baseline quality of code written by less experienced developers.
- Faster debugging - pasting an error or stack trace to an AI assistant frequently narrows down the root cause faster than manual searching.
- Supports learning - explanations that accompany AI generated code can help students understand unfamiliar patterns.

Risks and Limitations

- Hallucinated code - AI models can generate plausible looking but incorrect code, invented method/library names that do not exist, or subtly wrong logic that compiles but behaves incorrectly.
- Over reliance - excessive dependence on AI suggestions can reduce a developer's own problem solving practice and weaken their understanding of the underlying logic.
- Security issues – AI generated code can reproduce insecure patterns seen in its training data if not carefully reviewed.
- Licensing and originality concerns - generated code may closely resemble copyrighted training examples, raising intellectual property questions in commercial contexts.
- False confidence - code that "looks" professional (proper naming, comments) can mask logic errors that a reviewer assumes are correct because of its polished appearance.
- Context limitations - AI assistants may not fully understand a large, pre existing codebase, leading to suggestions that are locally sensible but globally inconsistent with the project's architecture.

Practical Implementation

The chosen project is a Library Management System, a moderately complex problem suggested in the assignment brief. It focuses on three main entities books, members, and loans and allows issuing and returning books through a simple console menu.

As required, the system was implemented twice:

Version A – built using a traditional programming approach, without AI assistance. It follows the same procedural, array based style used in earlier IT3003 lab work

Version B – developed with the support of an AI coding assistant (Claude), applying proper object oriented design principles, exception handling, and file persistence.

Both versions were compiled and executed using JDK 21.0.4 from the command line, and their console sessions were documented with screenshots included below.

Version A - Traditional Approach (LibrarySystemA.java)

```
import java.util.Scanner;

class Book {

    int id;
    String title;
    String author;
    boolean available;

    Book(int a, String b, String c) {
        id = a;
        title = b;
        author = c;
        available = true;
    }
}

class Member {

    int id;
    String name;

    Member(int a, String b) {
        id = a;
        name = b;
    }
}
```

```

    }
}

public class LibrarySystemA {

    static Scanner sc = new Scanner(System.in);

    static Book[] books = new Book[50];
    static Member[] members = new Member[50];

    static int[] issueBook = new int[50];
    static int[] issueMember = new int[50];

    static int bookCount = 0;
    static int memberCount = 0;
    static int issueCount = 0;

    public static void main(String[] args) {

        int choice = 0;

        while (choice != 7) {

            System.out.println();
            System.out.println("===== LIBRARY MANAGEMENT SYSTEM =====");
            System.out.println("1. Add Book");
            System.out.println("2. Add Member");
            System.out.println("3. Issue Book");
            System.out.println("4. Return Book");
            System.out.println("5. Display Books");
            System.out.println("6. Display Members");
            System.out.println("7. Exit");
            System.out.print("Enter Choice : ");

            choice = sc.nextInt();

            if (choice == 1) {

                sc.nextLine();

                System.out.print("Book ID : ");
                int x = sc.nextInt();
                sc.nextLine();

                System.out.print("Book Title : ");
                String y = sc.nextLine();

                System.out.print("Author : ");
                String z = sc.nextLine();
            }
        }
    }
}

```

```

        books[bookCount] = new Book(x, y, z);
        bookCount = bookCount + 1;

        System.out.println("Book Added");

    } else if (choice == 2) {

        sc.nextLine();

        System.out.print("Member ID : ");
        int x = sc.nextInt();
        sc.nextLine();

        System.out.print("Member Name : ");
        String y = sc.nextLine();

        members[memberCount] = new Member(x, y);
        memberCount = memberCount + 1;

        System.out.println("Member Added");

    } else if (choice == 3) {

        System.out.print("Book ID : ");
        int b = sc.nextInt();

        System.out.print("Member ID : ");
        int m = sc.nextInt();

        boolean found = false;

        for (int i = 0; i < bookCount; i = i + 1) {

            if (books[i].id == b) {

                found = true;

                if (books[i].available == true) {

                    books[i].available = false;

                    issueBook[issueCount] = b;
                    issueMember[issueCount] = m;
                    issueCount = issueCount + 1;

                    System.out.println("Book Issued");

                } else {

```

```

        System.out.println("Book Already Issued");
    }
}

if (found == false) {
    System.out.println("Book Not Found");
}

} else if (choice == 4) {

    System.out.print("Book ID : ");
    int b = sc.nextInt();

    boolean found = false;

    for (int i = 0; i < bookCount; i = i + 1) {

        if (books[i].id == b) {

            books[i].available = true;
            found = true;
        }
    }

    if (found == true) {
        System.out.println("Book Returned");
    } else {
        System.out.println("Book Not Found");
    }
}

} else if (choice == 5) {

    System.out.println();
    System.out.println("----- Book List -----");

    if (bookCount == 0) {

        System.out.println("No Books");

    } else {

        for (int i = 0; i < bookCount; i = i + 1) {

            System.out.println("Book ID : " + books[i].id);
            System.out.println("Title : " + books[i].title);
            System.out.println("Author : " + books[i].author);
            System.out.println("Available : " + books[i].available);
        }
    }
}

```

```

        System.out.println();
    }
}

} else if (choice == 6) {

    System.out.println();
    System.out.println("----- Member List -----");

    if (memberCount == 0) {

        System.out.println("No Members");

    } else {

        for (int i = 0; i < memberCount; i = i + 1) {

            System.out.println("Member ID : " + members[i].id);
            System.out.println("Member Name : " + members[i].name);
            System.out.println();
        }
    }

} else if (choice == 7) {

    System.out.println("Program End");

} else {

    System.out.println("Wrong Choice");
}

}

sc.close();
}
}

```

(Full source: VA/LibrarySystemA.java in the GitHub repository.)

Console output - adding books/members and issuing a loan:

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Exit
Enter Choice :
1

Book ID :
001

Book Title :
Harry potter

Author :
JK

Book Added

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Exit
Enter Choice :
2

Member ID :
0001

Member Name :
Kaveesh

Member Added

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Exit
Enter Choice :
3

Book ID :
001

Member ID :
0001

Book Issued

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Exit
Enter Choice :
5

----- Book List -----
Book ID : 1
Title : Harry potter
Author : JK
Available : false
```

Figure A1: Version A console session (add book/member, issue book, list books).

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Exit
Enter Choice :
6

----- Member List -----
Member ID : 1
Member Name : Kaveesh
②
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Exit
Enter Choice :
4

Book ID :
001

Book Returned

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Exit
Enter Choice :
-

Book Returned

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Exit
Enter Choice :
5

----- Book List -----
Book ID : 1
Title : Harry potter
Author : JK
Available : true
③
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Exit
Enter Choice :
7

Program End
User program finished
```

Figure A2: Version A console session (list members, return book, exit).

Version B – AI Assisted Approach

Version B was developed with AI assistance and was decomposed into six focused classes like Book, Member, Loan, Library, LibraryException, and LibraryManagementSystemB (console entry point). The AI assistant was prompted to apply proper encapsulation, meaningful exception handling, input validation, javadoc, and simple file based persistence (books.txt / members.txt) so that data survives between runs.

```
package library;

import java.util.InputMismatchException;
import java.util.List;
import java.util.Scanner;

public class LibraryManagementSystemB {

    private static final Scanner sc = new Scanner(System.in);
    private static final Library library = new Library();

    public static void main(String[] args) {
        library.loadFromFile();
        boolean running = true;

        while (running) {
            printMenu();
            int choice = readInt("Enter choice: ");

            try {
                switch (choice) {
                    case 1 -> addBook();
                    case 2 -> addMember();
                    case 3 -> issueBook();
                    case 4 -> returnBook();
                    case 5 -> displayBooks();
                    case 6 -> displayMembers();
                    case 7 -> displayActiveLoans();
                    case 8 -> {
                        library.saveToFile();
                        System.out.println("Data saved. Exiting system. Goodbye!");
                        running = false;
                    }
                    default -> System.out.println("Invalid choice. Please select 1-8.");
                }
            } catch (LibraryException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}
```

```

        sc.close();
    }

    private static void printMenu() {
        System.out.println("\n===== LIBRARY MANAGEMENT SYSTEM (Version B - AI-Assisted) =====");
        System.out.println("1. Add Book");
        System.out.println("2. Add Member");
        System.out.println("3. Issue Book");
        System.out.println("4. Return Book");
        System.out.println("5. Display Books");
        System.out.println("6. Display Members");
        System.out.println("7. Display Active Loans");
        System.out.println("8. Save & Exit");
    }

    private static void addBook() throws LibraryException {
        int id = readInt("Book ID: ");
        String title = readNonEmptyLine("Title: ");
        String author = readNonEmptyLine("Author: ");
        String isbn = readNonEmptyLine("ISBN: ");

        library.addBook(new Book(id, title, author, isbn));
        System.out.println("Book added successfully.");
    }

    private static void addMember() throws LibraryException {
        int id = readInt("Member ID: ");
        String name = readNonEmptyLine("Name: ");
        String email = readNonEmptyLine("Email: ");

        library.addMember(new Member(id, name, email));
        System.out.println("Member added successfully.");
    }

    private static void issueBook() throws LibraryException {
        int bookId = readInt("Book ID to issue: ");
        int memberId = readInt("Member ID: ");
        Loan loan = library.issueBook(bookId, memberId);
        System.out.println("Book issued. Due date: " + loan.getDueDate());
    }

    private static void returnBook() throws LibraryException {
        int bookId = readInt("Book ID to return: ");
        Loan loan = library.returnBook(bookId);
        double fine = loan.calculateFine();
        System.out.println("Book returned.");
        if (fine > 0) {
            System.out.printf("This book was returned late. Fine due: LKR %.2f%n", fine);
        }
    }

```

```

    }
}

private static void displayBooks() {
    List<Book> books = library.getBooks();
    System.out.println("\n-- Book Catalogue (" + books.size() + ") --");
    if (books.isEmpty()) {
        System.out.println("No books registered yet.");
    }
    for (Book b : books) {
        System.out.println(b);
    }
}

private static void displayMembers() {
    List<Member> members = library.getMembers();
    System.out.println("\n-- Members (" + members.size() + ") --");
    if (members.isEmpty()) {
        System.out.println("No members registered yet.");
    }
    for (Member m : members) {
        System.out.println(m);
    }
}

private static void displayActiveLoans() {
    List<Loan> active = library.getActiveLoans();
    System.out.println("\n-- Active Loans (" + active.size() + ") --");
    if (active.isEmpty()) {
        System.out.println("No books are currently on loan.");
    }
    for (Loan l : active) {
        System.out.println(l);
    }
}

// ----- Input helpers (defensive input handling) -----

private static int readInt(String prompt) {
    while (true) {
        System.out.print(prompt);
        try {
            int value = Integer.parseInt(sc.nextLine().trim());
            return value;
        } catch (NumberFormatException e) {
            System.out.println("Please enter a valid whole number.");
        }
    }
}

```

```

private static String readNonEmptyLine(String prompt) {
    while (true) {
        System.out.print(prompt);
        String value = sc.nextLine().trim();
        if (!value.isEmpty()) {
            return value;
        }
        System.out.println("This field cannot be empty.");
    }
}
}

```

```

package library;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class Loan {

    private static final int LOAN_PERIOD_DAYS = 14;
    private static final double FINE_PER_DAY = 20.0;

    private final Book book;
    private final Member member;
    private final LocalDate issueDate;
    private final LocalDate dueDate;
    private LocalDate returnDate;

    public Loan(Book book, Member member, LocalDate issueDate) {
        this.book = book;
        this.member = member;
        this.issueDate = issueDate;
        this.dueDate = issueDate.plusDays(LOAN_PERIOD_DAYS);
    }

    public Book getBook() {
        return book;
    }

    public Member getMember() {
        return member;
    }

    public LocalDate getDueDate() {

```

```

        return dueDate;
    }

    public LocalDate getReturnDate() {
        return returnDate;
    }

    public boolean isReturned() {
        return returnDate != null;
    }

    public void markReturned(LocalDate returnDate) {
        this.returnDate = returnDate;
    }

    /** Calculates the fine owed based on how late the book was (or currently is). */
    public double calculateFine() {
        LocalDate compareDate = isReturned() ? returnDate : LocalDate.now();
        long daysLate = ChronoUnit.DAYS.between(dueDate, compareDate);
        return daysLate > 0 ? daysLate * FINE_PER_DAY : 0.0;
    }

    @Override
    public String toString() {
        String status = isReturned() ? "Returned on " + returnDate : "Due on " + dueDate;
        double fine = calculateFine();
        String fineText = fine > 0 ? String.format(" | Fine: LKR %.2f", fine) : "";
        return String.format("Book #%-> Member #%- | Issued: %s | %s%s",
            book.getId(), member.getId(), issueDate, status, fineText);
    }
}

```

(Full source: VB/LibrarySystemB.java in the GitHub repository.)

Console output - adding books/members and issuing a loan:

```
===== LIBRARY MANAGEMENT SYSTEM (Version B - AI-Assisted) =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Display Active Loans
8. Save & Exit
Enter choice: 1
Book ID: 001
Title: Harry Potter
Author: JK
ISBN: 43346939
Book added successfully.

===== LIBRARY MANAGEMENT SYSTEM (Version B - AI-Assisted) =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Display Active Loans
8. Save & Exit
Enter choice: 2
Member ID: 0001
Name: Kaveesh
Email: kaveesh@gmail.com
Member added successfully.

===== LIBRARY MANAGEMENT SYSTEM (Version B - AI-Assisted) =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Display Active Loans
8. Save & Exit
Enter choice: 3

PS C:\Users\VASUS\Desktop\ass2\VB> java -cp bin library.LibraryManagementSystemB
7. Display Active Loans
8. Save & Exit
Enter choice: 3
Book ID to issue: 001
Member ID: 0001
Book issued. Due date: 2026-07-28

===== LIBRARY MANAGEMENT SYSTEM (Version B - AI-Assisted) =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Display Active Loans
8. Save & Exit
Enter choice: 5

-- Book Catalogue (1) --
[1] "Harry Potter" by JK (ISBN: 43346939) - On Loan

===== LIBRARY MANAGEMENT SYSTEM (Version B - AI-Assisted) =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Display Active Loans
8. Save & Exit
Enter choice: 6

-- Members (1) --
[1] Kaveesh (kaveesh@gmail.com)
```

Figure B1: Version B console session (add book/member, issue, duplicate issue error handled, list book/members/loans).

```

===== LIBRARY MANAGEMENT SYSTEM (Version B - AI-Assisted) =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Display Active Loans
8. Save & Exit
Enter choice: 7

-- Active Loans (1) --
Book #1 -> Member #1 | Issued: 2026-07-14 | Due on 2026-07-28

===== LIBRARY MANAGEMENT SYSTEM (Version B - AI-Assisted) =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Display Active Loans
8. Save & Exit
Enter choice: 4
Book ID to return: 001
Book returned.

===== LIBRARY MANAGEMENT SYSTEM (Version B - AI-Assisted) =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Display Active Loans
8. Save & Exit
Enter choice: 5

-- Book Catalogue (1) --
[1] "Harry Potter" by JK (ISBN: 43346930) - Available

```

```

===== LIBRARY MANAGEMENT SYSTEM (Version B - AI-Assisted) =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Display Active Loans
8. Save & Exit
Enter choice: 7

-- Active Loans (0) --
No books are currently on loan.

===== LIBRARY MANAGEMENT SYSTEM (Version B - AI-Assisted) =====
1. Add Book
2. Add Member
3. Issue Book
4. Return Book
5. Display Books
6. Display Members
7. Display Active Loans
8. Save & Exit
Enter choice: 8
Data saved. Exiting system. Goodbye!
PS C:\Users\VASU5\Desktop\lass2\VB>

```

Figure B2: Version B console session (return book, updated catalogue, active loans, save & exit).

Comparison of Version A and Version B

Criterion	Version A (Traditional)	Version B (AI Assisted)
Development time	~40 minutes for a minimal feature set (arrays, no persistence, no fines).	~70 minutes for a much larger feature set (persistence, fines, custom exceptions, javadoc) - AI generated boilerplate (getters, serialization, javadoc) in seconds, leaving more time for reviewing logic.
Code quality	Functional but repetitive if/else chain, public mutable fields, no encapsulation, magic numbers (array size 50).	Encapsulated fields with getters/setters, switch expressions, a dedicated Loan/LibraryException type, consistent javadoc and formatting.
Error handling	None. Non numeric menu input throws an uncaught InputMismatchException and crashes the program, more than 50 books/members causes an ArrayIndexOutOfBoundsException.	readInt()/readNonEmptyLine() helpers reject bad input without crashing, LibraryException surfaces domain errors (duplicate IDs, book already on loan, not found) as a friendly message instead of a stack trace.
Readability & maintainability	Single ~140 line file, everything static and adding a feature such as fines would require touching the monolithic main() method in several places.	Split into 6 focused classes. adding a feature such as loan history mainly touches Library and Loan, following single responsibility principle.

UML Diagrams

The following diagrams model the Library Management System. They were produced to improve understanding of the selected problem and the system design before and while implementing Version A and B.

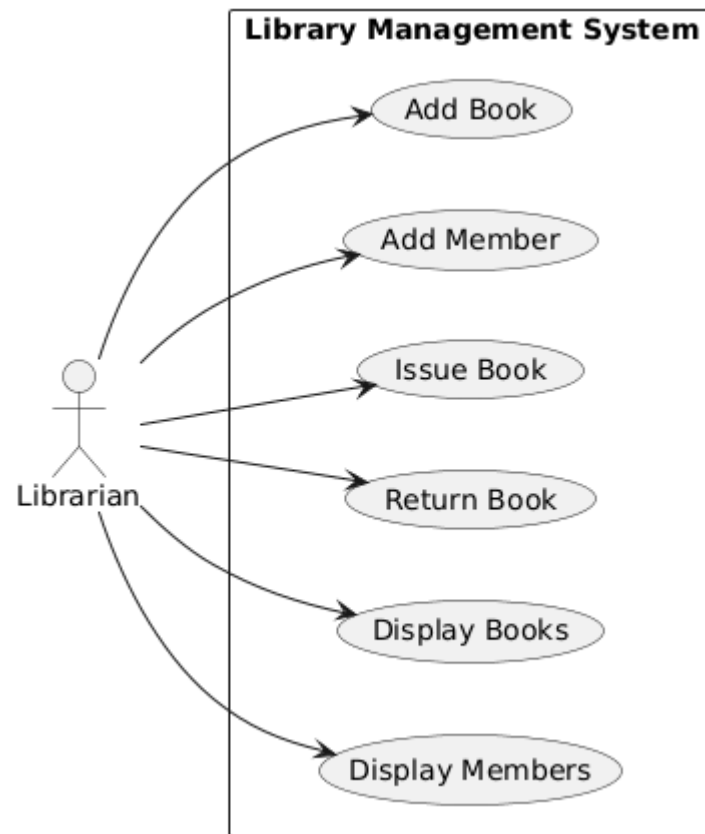


Figure 1: Use Case Diagram - the Librarian actor and the six supported use cases.

Class Diagram - Library Management System (Version B)

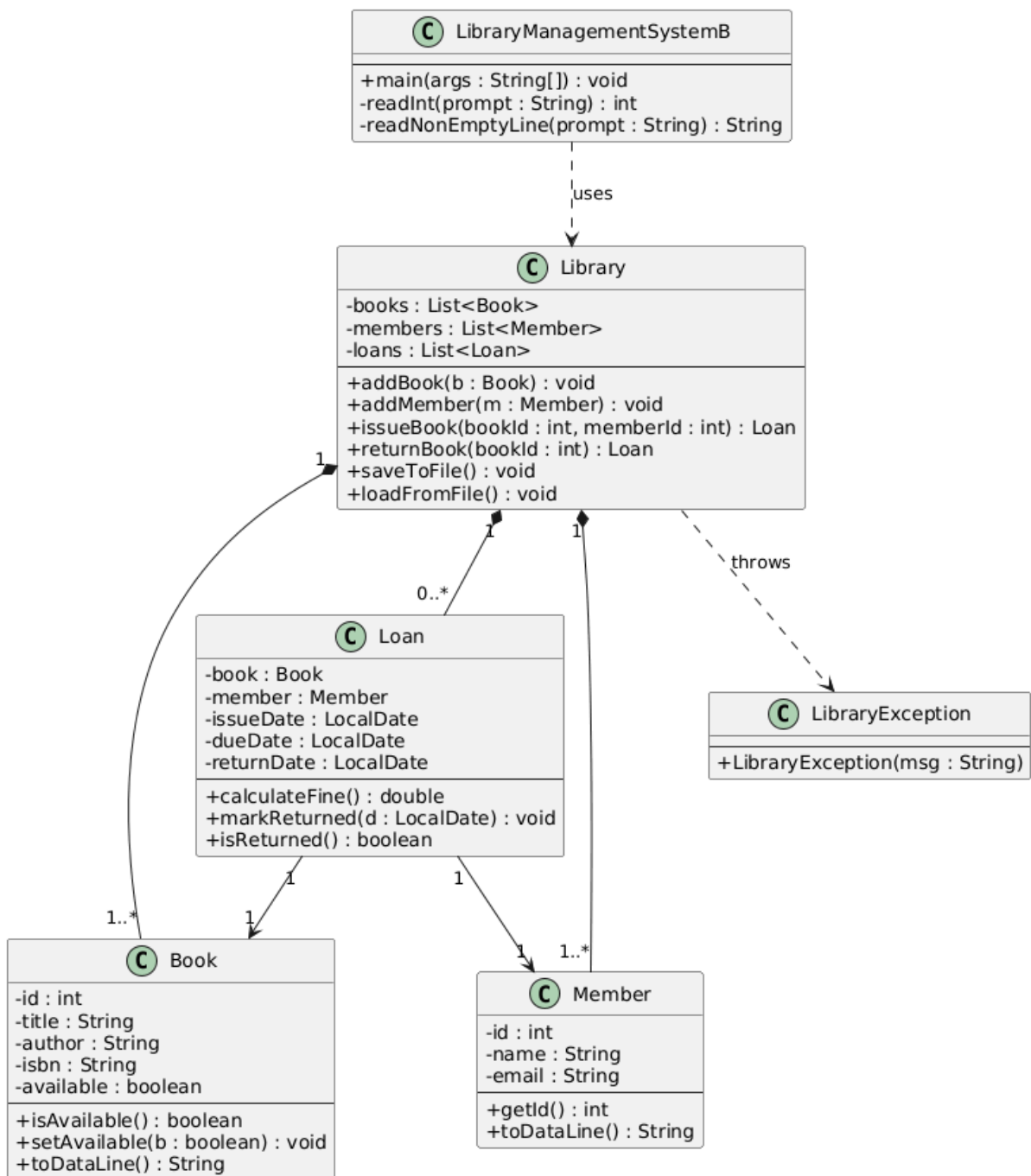


Figure 2: Class Diagram - Book, Member, Loan, Library, LibraryException and the console entry point.

Activity Diagram - Issue Book Process (Version B)

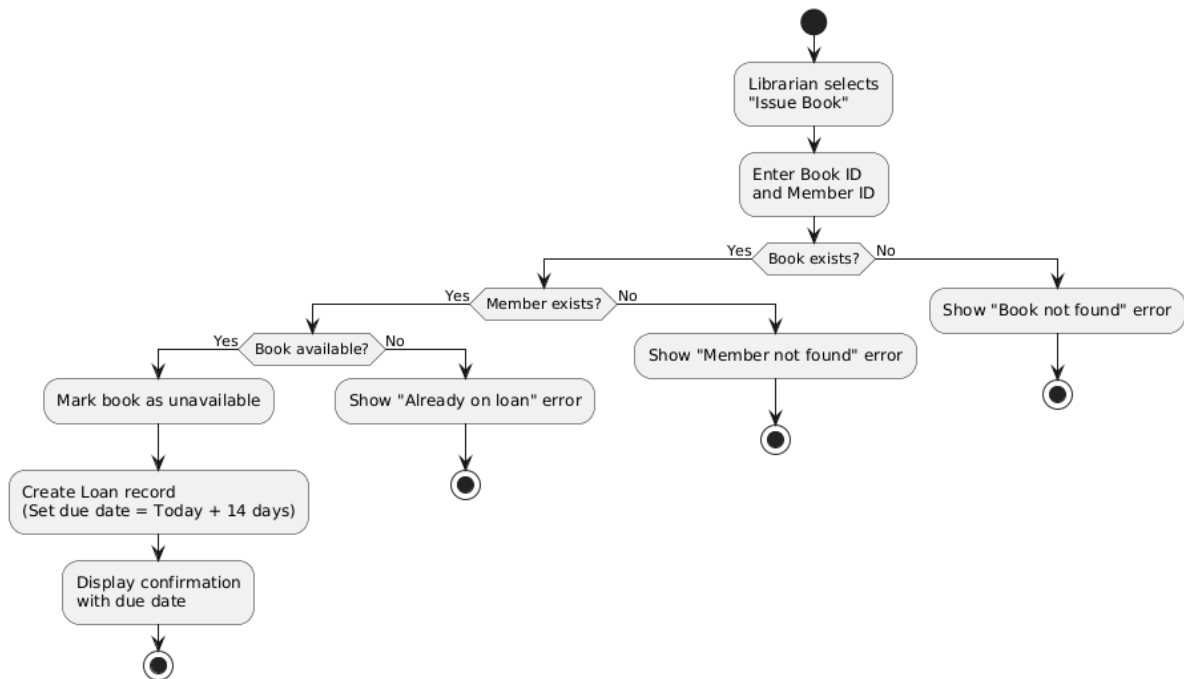


Figure 3: Activity Diagram - the Issue Book workflow, including the three validation checks.

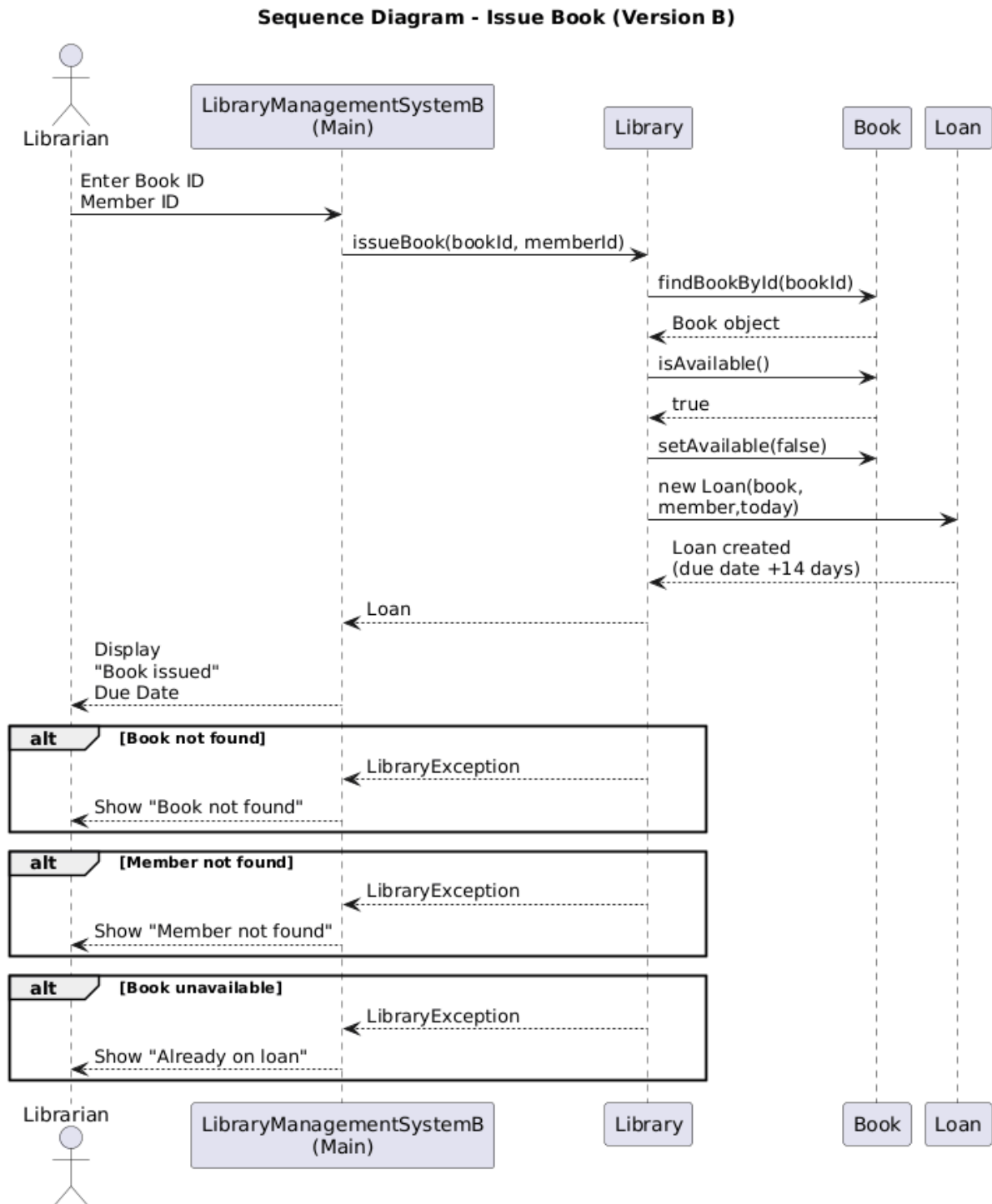


Figure 4: Sequence Diagram - object interactions for a successful Issue Book call.

AI Prompts Used During Development

The following prompts were used with the AI coding assistant to produce Version B.

- "write a java console based library management system with proper oop design. book, member and loan classes, custom exception handling for invalid operations, and simple text file persistence."
- "add a loan class that calculates an overdue fine based on a 14 day loan period and a fixed per day fine rate."
- "replace the raw scanner.nextint() calls with input helper methods that re prompt on invalid input instead of crashing."
- "add javadoc comments explaining the purpose of each class and method."

Critical Analysis

How AI Changed the Coding Approach

Building Version B shifted the workflow from writing every statement by hand to specifying intent through prompts and then reviewing, compiling, and testing the result. Design decisions were made up front in natural language, and the AI assistant translated them into working Java. This moved the developer's effort away from syntax and boilerplate and toward specification, review, and verification closer to a design or review role than a pure implementation role.

Did AI Improve or Reduce Understanding?

Both effects were observed. AI improved understanding in areas the student may not have used confidently before, such as custom checked exceptions, `Optional<T>` for null safe lookups, and simple file persistence with `try with resources` the accompanying explanations acted as a mini tutorial. At the same time, understanding can be reduced if a developer accepts generated code without tracing through it. for example, the fine calculation in `Loan.calculateFine()` (`LocalDate.now()` vs. a stored `returnDate`) is easy to misuse if its exact semantics are not read carefully. The net effect depends heavily on whether the developer treats AI output as a first draft to be verified or as a finished answer to be trusted blindly.

When AI Should NOT Be Used

- When learning fundamentals for the first time - relying on AI to write introductory exercises undermines the purpose of the exercise, which is to build the our own problem solving ability.
- In safety critical or security sensitive code (authentication, payment processing, medical or financial systems) without rigorous human review, since hallucinated or subtly incorrect logic can have serious consequences.
- When the required license/IP terms of the codebase are strict and the provenance of AI generated snippets cannot be verified.
- In high stakes assessments explicitly designed to test unaided individual ability, where AI use may violate academic integrity rules.
- When a task depends on deep, project specific context that the AI assistant has not been given suggestions in this situation are likely to be plausible but wrong.

Ethical Considerations in AI Generated Code

- Attribution and honesty - students and developers should disclose when and how AI tools were used, as done in this report's transparency note, rather than presenting AI output as unaided personal work.
- Accountability - the human developer remains responsible for the correctness, security, and behaviour of code they submit or ship, regardless of whether AI generated part of it.
- Bias and training data provenance - AI models can reproduce insecure or low quality patterns that were common in their training data, so uncritical acceptance can propagate bad practice at scale.
- Academic integrity - using AI to complete graded coursework may be restricted or require disclosure depending on institutional policy. this should always be checked before submission.
- Data privacy - pasting proprietary or sensitive code into a third-party AI service can expose it outside the organisation's control, which is a governance concern in commercial settings.

Conclusion

This assignment implemented the same Library Management System twice. once in a traditional, manually styled approach and once with AI assistance to directly compare their development time, code quality, error handling, and maintainability. The AI assisted version consistently produced better structured, more defensive code and a richer feature set for a modest increase in development time, largely because the AI assistant absorbed the cost of writing repetitive scaffolding.

Final opinion: AI assisted programming is a genuine productivity and quality multiplier for developers who already understand the fundamentals and use it to accelerate design decisions they can verify as demonstrated by Version B's cleaner architecture and defensive error handling. In programming education specifically, however, AI should be introduced carefully over reliance during the early learning stages risks weakening the very skills that let a developer evaluate AI output critically later on. The most sustainable path, for both students and industry practitioners, is to treat AI as a reviewed collaborator rather than an unquestioned author always compiling, testing, and reading generated code before trusting it, exactly as was done for version B in this report.

References

GitHub, Inc. (2024) GitHub Copilot Documentation. Available at: <https://docs.github.com/en/copilot> (Accessed: 3 July 2026).

OpenAI (2024) ChatGPT Documentation and Usage Policies. Available at: <https://help.openai.com> (Accessed: 3 July 2026).

Anthropic (2025) Claude Documentation. Available at: <https://docs.claude.com> (Accessed: 3 July 2026).

Martin, R.C. (2008) Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ: Prentice Hall.

Oracle (2024) The Java Tutorials - Exceptions. Available at: <https://docs.oracle.com/javase/tutorial/essential/exceptions/> (Accessed: 3 July 2026).

Appendix: GitHub Repository Submission

The full source code (both versions), documentation, project files and commit history for this assignment are hosted at the following GitHub repository:

Repository link: <https://github.com/mkpfd/ass02.git>

The repository contains:

- VA - the traditional, manually styled implementation.
- VB- the AI assisted implementation (Java package library).
- Diagrams - the four UML diagrams (PNG) embedded in this report.
- Outputs - the console session captures embedded in this report.
- Report - this report